

Yönetici Özeti

Bölüm 1 / 10

Bu belge, Türkiye Yapay Zeka Akademisi için hazırlanmış örnek bir kurum içi yapay zeka uygulama değerlendirme dokümanıdır. Amaç, on-prem çalışan büyük dil modellerinin belge özetleme ve karşılaştırma senaryolarında nasıl kullanılabileceğini sade bir dille açıklamaktır.

Kurumlar yapay zeka uygulamalarını kullanırken hız, maliyet, gizlilik ve kalite arasında denge kurmak zorundadır. Cloud LLM servisleri hızlı başlamak için güçlü bir seçenek sunarken, on-prem LLM yaklaşımı özellikle hassas belge ve kurum içi veri senaryolarında daha fazla kontrol sağlar.

Bu çalışma kapsamında yerel sunucuda çalışan bir büyük dil modeli, vLLM ile servis edilecek ve Python uygulaması üzerinden belge özetleme akışı kurulacaktır. İlk aşamada tek bir PDF belgesi okunacak, metni modele gönderilecek ve modelden profesyonel bir özet üretmesi istenecektir.

Belgenin ana önerisi, eğitim ortamında küçük ve yönetilebilir bir modelle başlanması, sistemin çalıştığının görülmesi ve daha sonra daha büyük modellere geçilmesidir. Böylece öğrenciler hem teknik altyapıyı hem de uygulama mantığını adım adım öğrenebilir.

Problem Tanımı

Bölüm 2 / 10

Kurumlar çok sayıda rapor, sözleşme, teklif, toplantı notu ve teknik doküman üretir. Bu belgelerin tamamını manuel olarak okumak zaman alır ve kritik bilgilerin gözden kaçmasına neden olabilir. Özellikle kısa sürede karar verilmesi gereken durumlarda belge özetleme sistemleri ciddi verimlilik sağlar.

Klasik arama sistemleri belgelerde kelime bulmak için faydalıdır; ancak belgenin ana fikrini çıkarmak, önemli noktaları ayırmak veya iki belge arasındaki farkları yorumlamak için yeterli olmayabilir. Büyük dil modelleri bu noktada metni anlamlandırma ve doğal dille çıktı üretme becerileriyle öne çıkar.

Bu projede hedef, kullanıcının verdiği PDF belgesini yerel çalışan bir modelle analiz etmek ve belgeyi kısa, düzenli, anlaşılır bir özet haline getirmektir. Çıktı insan tarafından okunabilir olacak, ancak ilerleyen aşamalarda JSON gibi yapılandırılmış formatlara da dönüştürülebilecektir.

Özetleme uygulaması temel bir kullanım senaryosu gibi görünse de, arka planda token kullanımı, context window, inference süresi, model seçimi ve çıktı kalitesi gibi birçok teknik konuya temas eder. Bu nedenle eğitim için uygun ve öğretici bir örnektir.

Cloud ve On-Prem Yaklaşımları

Bölüm 3 / 10

Cloud LLM servisleri, OpenAI, Gemini, Anthropic ve benzeri sağlayıcılar üzerinden API ile kullanılan büyük dil modelleridir. Bu yaklaşımda kurulum hızlıdır, güçlü modellere erişim kolaydır ve altyapı yönetimi büyük ölçüde sağlayıcı tarafından yapılır.

On-prem LLM yaklaşımında ise model kurumun kendi sunucusunda veya kendi kontrol ettiği altyapıda çalıştırılır. Bu durumda veri kurum içinde kalır, model seçimi ve servisleme süreci daha fazla kontrol edilebilir, ancak donanım ve bakım sorumluluğu kurumun kendisine geçer.

Cloud tarafı hızlı prototipleme, demo ve düşük operasyon yükü için uygundur. On-prem tarafı ise hassas veri, kurum içi belge analizi, özel güvenlik politikaları ve dış servise veri gönderilmemesi gereken durumlarda daha anlamlı hale gelir.

Bu belgede kullanılan yaklaşım on-prem taraftadır. Yani PDF belgesi dış API servislerine gönderilmeden, yerel makinede çalışan model üzerinden özetlenecektir. Bu sayede veri akışı daha kontrollü bir ortamda tutulur.

On-Prem LLM Mimarisi

Bölüm 4 / 10

On-prem LLM mimarisinde temel akış kurum içi veriyle başlar. Kullanıcı bir belge veya metin sağlar. Bu veri yerel sunucuda çalışan büyük dil modeline iletilir. Model inference sürecinde metni işler ve çıktı olarak özet, analiz ya da karşılaştırma sonucu üretir.

Bu yapıda model tek başına yeterli değildir. Modelin servis edilebilmesi için vLLM gibi bir çıkarım motoruna ihtiyaç duyulur. vLLM, büyük dil modellerini daha verimli çalıştırmak, GPU belleğini daha iyi kullanmak ve aynı anda daha fazla isteği yönetmek için tercih edilir.

Örnek mimaride Docker Desktop, Windows üzerinde container çalıştırmak için kullanılır. Docker arka planda WSL2 desteğiyle Linux tabanlı ortam sağlar. vLLM Docker container içinde çalışır ve model localhost üzerinden erişilebilir hale gelir.

Python uygulaması bu yerel servise istek gönderir. Böylece kullanıcı açısından süreç basit görünür: PDF dosyası seçilir, Python dosyası çalıştırılır, model belgeyi okur ve özet döndürür. Ancak arka planda Docker, vLLM, model dosyaları ve GPU birlikte çalışır.

Model Seçim Kriterleri

Bölüm 5 / 10

On-prem model seçerken yalnızca modelin popüler olmasına bakmak yeterli değildir. Model ailesi, parametre sayısı, context window kapasitesi, Türkçe performansı, lisans koşulları, donanım ihtiyacı ve hız-kalite dengesi birlikte değerlendirilmelidir.

Gemma, Qwen, Llama ve GPT-OSS gibi farklı model aileleri farklı güçlü yönlerle sahip olabilir. Bazı modeller daha hafif ve hızlı çalışırken, bazıları daha uzun bağlam veya daha güçlü analiz yeteneği sunabilir. Bu nedenle model seçimi mutlaka proje ihtiyacına göre yapılmalıdır.

Parametre sayısı arttıkça model genellikle daha güçlü hale gelir; ancak daha fazla VRAM ve işlem gücü ister. Küçük modeller hızlı ve erişilebilir olabilir, fakat karmaşık analizlerde sınırlı kalabilir. Büyük modeller daha kaliteli sonuç verebilir, ancak maliyet ve gecikme artabilir.

Belge özetleme projesinde context window önemli bir kriterdir. PDF metni modele tek seferde gönderilecekse, modelin bu metni sığdıracak bağlam kapasitesine sahip olması gerekir. Uzun belgelerde gerekirse metni bölme stratejileri kullanılabilir; ancak bu örnekte sade kalmak için kısa bir PDF seçilmiştir.

Donanım ve Çalışma Ortamı

Bölüm 6 / 10

On-prem LLM çalıştırırken donanım uygunluğu kritik öneme sahiptir. GPU var mı, VRAM yeterli mi, model kaç milyar parametre, quantized model kullanılacak mı ve context window büyüdükçe bellek ihtiyacı artacak mı gibi sorular baştan cevaplanmalıdır.

GPU, inference sürecini önemli ölçüde hızlandırır. CPU üzerinde model çalıştırmak mümkün olsa da pratik uygulamalarda yanıt süresi çoğu zaman yavaş kalır. Özellikle 4B, 7B ve üzeri modellerde GPU kullanımı eğitim ve demo deneyimini daha sağlıklı hale getirir.

VRAM, modelin belleğe sığıp sığmayacağını belirleyen temel kaynaktır. Model belleğe sığmazsa CUDA out of memory hatası alınabilir. Bu durumda daha küçük model seçmek, context uzunluğunu azaltmak veya quantized model kullanmak gerekebilir.

Docker ve WSL2, Windows üzerinde Linux tabanlı container çalıştırmak için kullanılır. Bu sayede vLLM gibi Linux odaklı araçlar Windows bilgisayarda daha düzenli ve taşınabilir bir ortamda çalıştırılabilir.

vLLM Kullanım Mantığı

Bölüm 7 / 10

vLLM, büyük dil modellerini yüksek performansla çalıştırmak ve servis etmek için kullanılan açık kaynaklı bir çıkarım motorudur. Bu projede vLLM, yerel çalışan Gemma modelini servis haline getirmek için kullanılacaktır.

vLLM'in tercih edilmesinin temel nedeni performans ve verimliliğidir. Daha iyi GPU bellek kullanımı, düşük gecikme, yüksek throughput ve daha fazla eşzamanlı istek yönetimi gibi avantajlar sunar. Bu avantajlar özellikle gerçek uygulama senaryolarında önemlidir.

Bu projede model, vLLM Docker container içinde çalışacaktır. Kullanıcı Windows tarafında Python kodunu çalıştıracak, Python kodu ise localhost üzerinde çalışan vLLM servisine istek gönderecektir. Model cevabı yine Python terminalinde gösterilecektir.

vLLM kurulumu ve model indirme süreci bir kez tamamlandıktan sonra aynı yapı farklı uygulamalar için tekrar kullanılabilir. Özetleme, karşılaştırma, soru-cevap ve belge analizi gibi görevler aynı temel servis üzerine kurulabilir.

PDF Özetleme Uygulaması

Bölüm 8 / 10

PDF özetleme uygulamasında kullanıcı terminal üzerinden bir PDF dosyası yolu girer. Python kodu pypdf kütüphanesiyle PDF içindeki metni okur ve bu metni vLLM üzerinde çalışan modele gönderir. Model, verilen metne sadık kalarak Türkçe bir özet üretir.

Bu uygulamada süreç özellikle sade tutulmuştur. Amaç, öğrencinin temel akışı görmesidir: PDF oku, metni modele gönder, özet al. Daha ileri seviye uygulamalarda uzun belgeler parçalara bölünebilir, parça özetleri birleştirilebilir veya çıktı JSON formatında üretilebilir.

Özetleme promptunda modelden belgenin ana amacını açıklaması, önemli noktaları madde madde yazması, gereksiz tekrarları çıkarması ve metinde olmayan bilgi eklememesi istenir. Bu kurallar, model çıktısının daha kontrollü ve işe yarar olmasını sağlar.

Model çıktısı insan tarafından kontrol edilmelidir. LLM çıktıları akıcı olabilir ancak her zaman doğru olmayabilir. Özellikle kurumsal karar süreçlerinde özetin belgeye sadık olup olmadığı ve önemli bilgileri kaçırıp kaçırmadığı kontrol edilmelidir.

Belge Karşılaştırma Senaryosu

Bölüm 9 / 10

Belge karşılaştırma, özetleme uygulamasının bir adım ileri versiyonudur. Burada amaç iki farklı belgenin ortak noktalarını, farklılıklarını, eksik veya çelişkili taraflarını model yardımıyla belirlemektir.

Örneğin iki teklif dosyası, iki sözleşme taslağı, iki eğitim planı veya iki teknik doküman karşılaştırılabilir. Modelden her iki belgeyi ayrı ayrı anlaması, ardından karşılaştırma sonucunu düzenli bir formatta sunması istenebilir.

Karşılaştırma uygulamasında context window daha önemli hale gelir. Çünkü modele aynı anda iki belge verilecektir. Belgeler kısa ise tek prompt yeterli olabilir; belgeler uzunsa parçalara bölme, özetleme veya RAG tabanlı yaklaşımlar gerekebilir.

Eğitimde bu senaryo, on-prem LLM kullanımının yalnızca sohbetten ibaret olmadığını gösterir. Kurumsal belge yönetimi, satın alma, hukuk, eğitim içerikleri ve teknik doküman analizleri gibi birçok gerçek iş akışına uyarlanabilir.

Sonuç ve Öneriler

Bölüm 10 / 10

On-prem LLM yaklaşımı, kurum içi belge analizi ve hassas veriyle çalışma senaryolarında güçlü bir seçenektir. Bu yaklaşım verinin kurum içinde kalmasını sağlar, model ve altyapı üzerinde daha fazla kontrol sunar ve özel iş akışlarına uyarlanabilir.

Buna karşılık on-prem kullanım donanım, kurulum, bakım ve model seçimi açısından dikkat ister. Küçük modellerle başlamak, sistemi çalıştırdıktan sonra kalite ve performans ihtiyacına göre daha büyük modellere geçmek en sağlıklı yaklaşımlardan biridir.

Bu örnek projede temel hedef, öğrencinin Docker, vLLM, yerel model, Python istemcisi ve PDF özetleme akışını uçtan uca görmesidir. Böylece teorik olarak anlatılan token, context window, inference ve yapılandırılmış çıktı gibi kavramlar gerçek uygulama içinde daha anlaşılır hale gelir.

Bir sonraki aşamada aynı altyapı belge karşılaştırma, structured output, değerlendirme kriterleri ve kurumsal asistan senaryolarına genişletilebilir. Bu yapı, eğitim içinde pratik ve tekrar kullanılabilir bir on-prem LLM temeli oluşturur.